

Let's look at the changes, one by one:

1. Bootstrap is the 'start point' of the Angular app. Let's make it empty. So, we do not have any Angular entry point for this app, as of now.
2. Let's put MyWebComponent into the entryComponents array. This means that now our Web component will be an entry point for the app.
3. Inject the Injector in the constructor; this will be used later in the `ngDoBootstrap` method given below.
4. A new method `ngBootstrap` is added. Following are the changes related to `ngBootstrap`:
 - a. `createCustomElement` from `@angular/elements` translates the vanilla Angular component to a Web component, along with its dependencies.
 - b. To register the newly created custom Web component to the browser, we are using the `customElements.define` method. The element is the `webComponent` and its handle is called `my-web-component`. Going forward, we shall use `my-web-component` to refer to this Web component.

Now, an important change needs to be made at the top level (i.e., inside the `angular-web-component` directory) in the `tsconfig.json` file. Let's change the target variable from `es5` to `es2015`. This is so that the browser supports custom Web components, natively.

That's all. Now let's build and test it!

Packaging the Web component

We need to package the Web component and all of its prerequisites into a wholesome JavaScript file. Let's now do it in the `package.json` file as follows.

 **Note:** If you're in Mac or Linux, instead of `jscat`, you can simply use inbuilt `cat`.

For Windows, one needs to install `jscat` (or merge it). Please install `jscat` as follows:

```
npm install jscat
```

File: `package.json` (added the following two lines)

```
"scripts": {
  .....
  "build-prod": "ng build --prod --output-hashing=none",
  "package": "jscat ./dist/angular-web-component/runtime.js
./dist/angular-web-component/polyfills.js ./dist/angular-web-
component/main.js > angular-web-component.js"
},
```

Now, to build up the `angular-web-component.js` file, let's

run the following command:

```
npm run build-prod && npm run package
```

This results in an `angular-web-component.js` file in the top-level root directory (i.e., inside the `angular-web-component` directory).

Testing it

Now it's the time to see your handiwork in action! A simple demonstration will suffice. Let the application continue to run (i.e., let `npm` continue to run). Let's change the `index.html` file as follows.

```
.....
<body>
  <my-web-component></my-web-component>
  <script type="text/javascript" src="../angular-web-
component.js">
  </script>
</body>
.....
```

Now you can see that the output shown in Figure 2 appears in the browser. Congrats – you've done it!

In this article, to tackle the evolving need for platform-agnostic front-end applications, we discussed the Web component that we could use. This makes the component reusable, framework-agnostic and interoperable across JavaScript frameworks. There are plenty of other evolving platforms too, which are specialised for building the Web components from the ground up, and are mostly FOSS. Google's Polymer `litElement` is one of the pioneers here. Others are `Stencil`, `Haunted`, etc. Overall, things are pretty much still evolving, so do keep watching this space.

 **Note:** The example code shown in this article can be found in github: <https://github.com/pradip-interra/angular-web-component>

END 

References

- [1] <https://angular.io/docs>
- [2] <https://jsonplaceholder.typicode.com>
- [3] <https://lit-element.polymer-project.org/>
- [4] <https://stenciljs.com/>
- [5] <https://github.com/matthewp/haunted>

By: Pradip Mukhopadhyay

The author has 19 years of experience across the stack – from low level systems programming to high level GUIs. He is a FOSS enthusiast, and currently works for NetApp, Bengaluru.